



智能计算系统

第七章

深度学习处理器架构

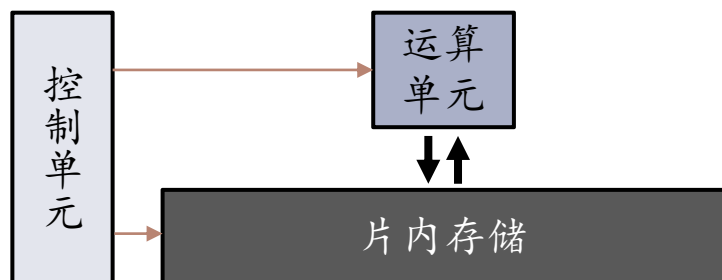
北京邮电大学 人工智能学院

何召锋、徐振博、项刘宇

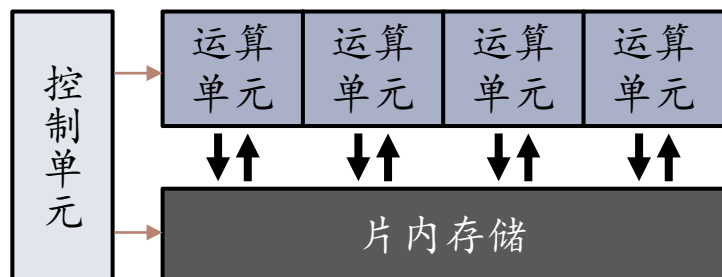
深度学习处理器 (DLP)

- 基本运算单元从**向量运算**扩展到**矩阵运算**

标量通用处理器



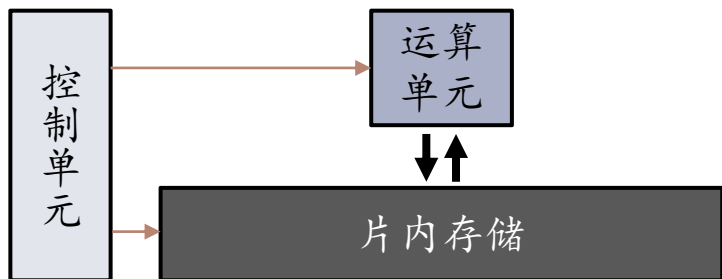
向量处理器



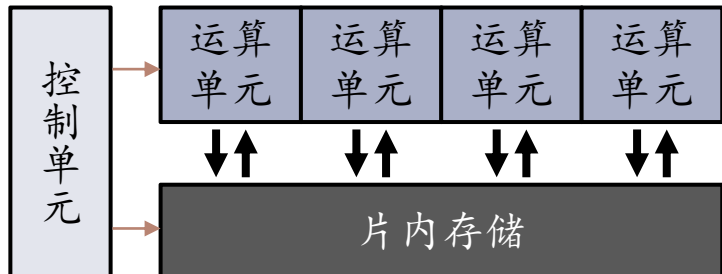
深度学习处理器 (DLP)

- 基本运算单元从**向量运算**扩展到**矩阵运算**

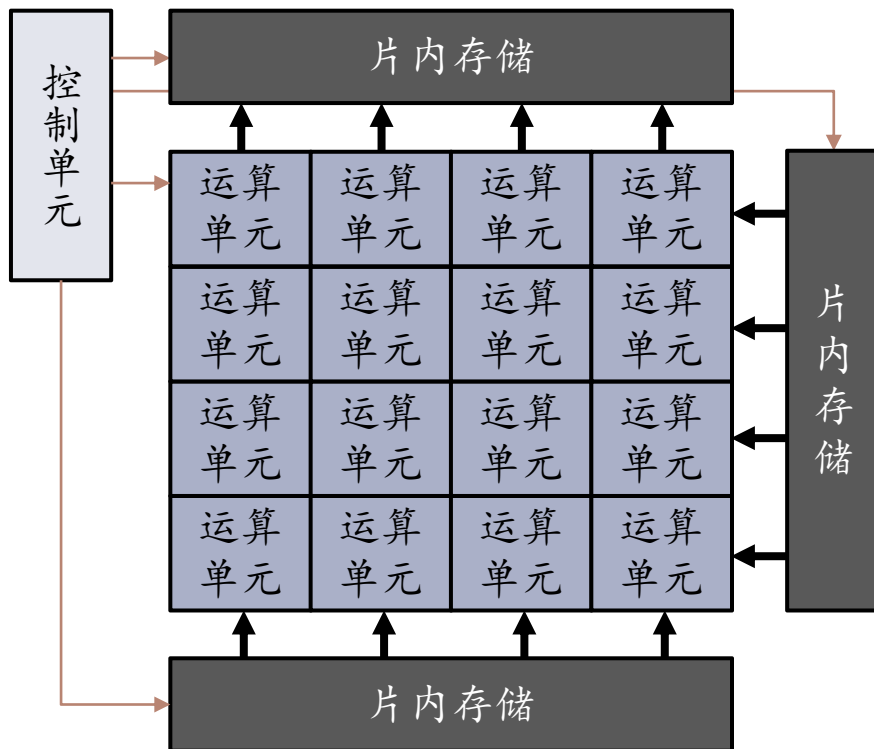
标量通用处理器



向量处理器

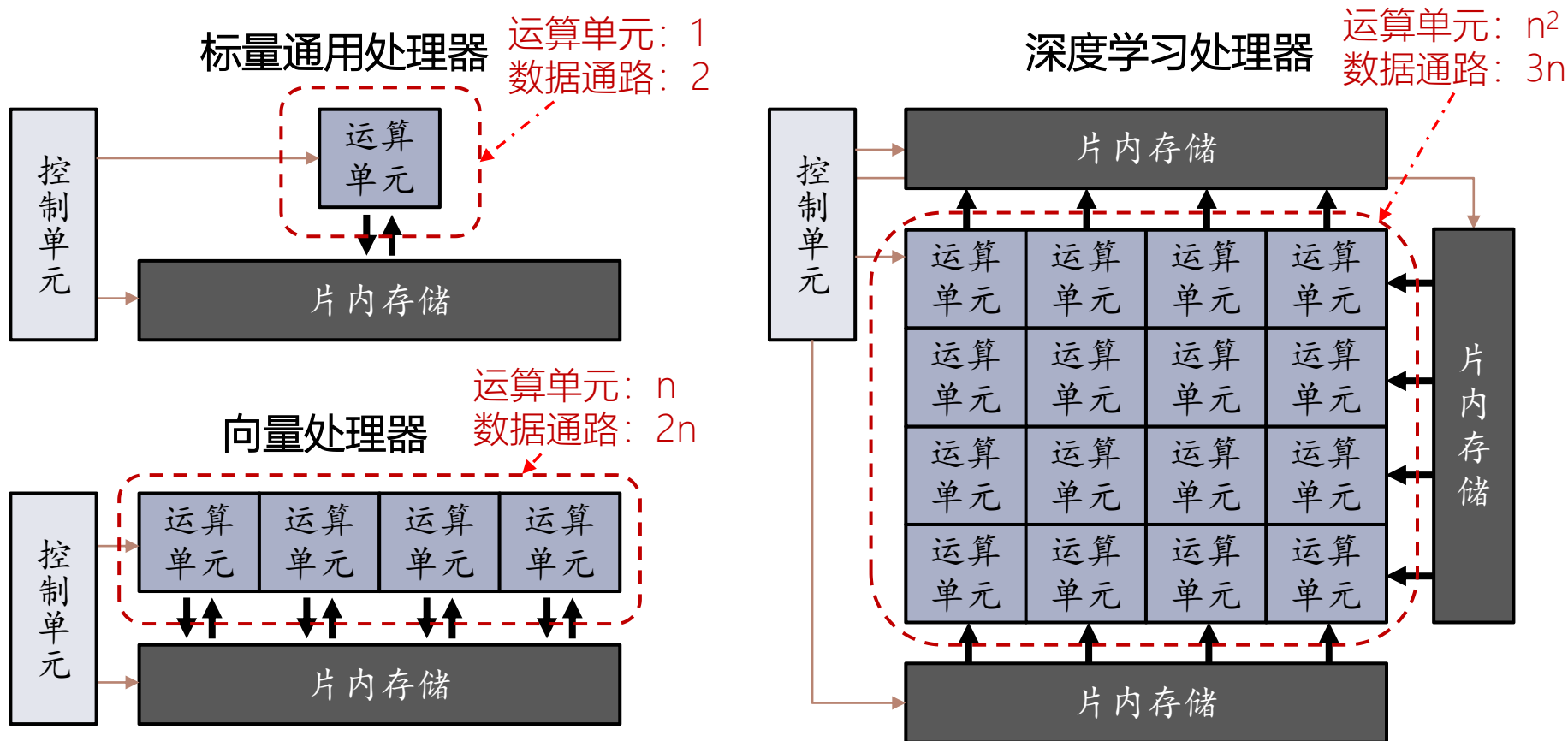


深度学习处理器



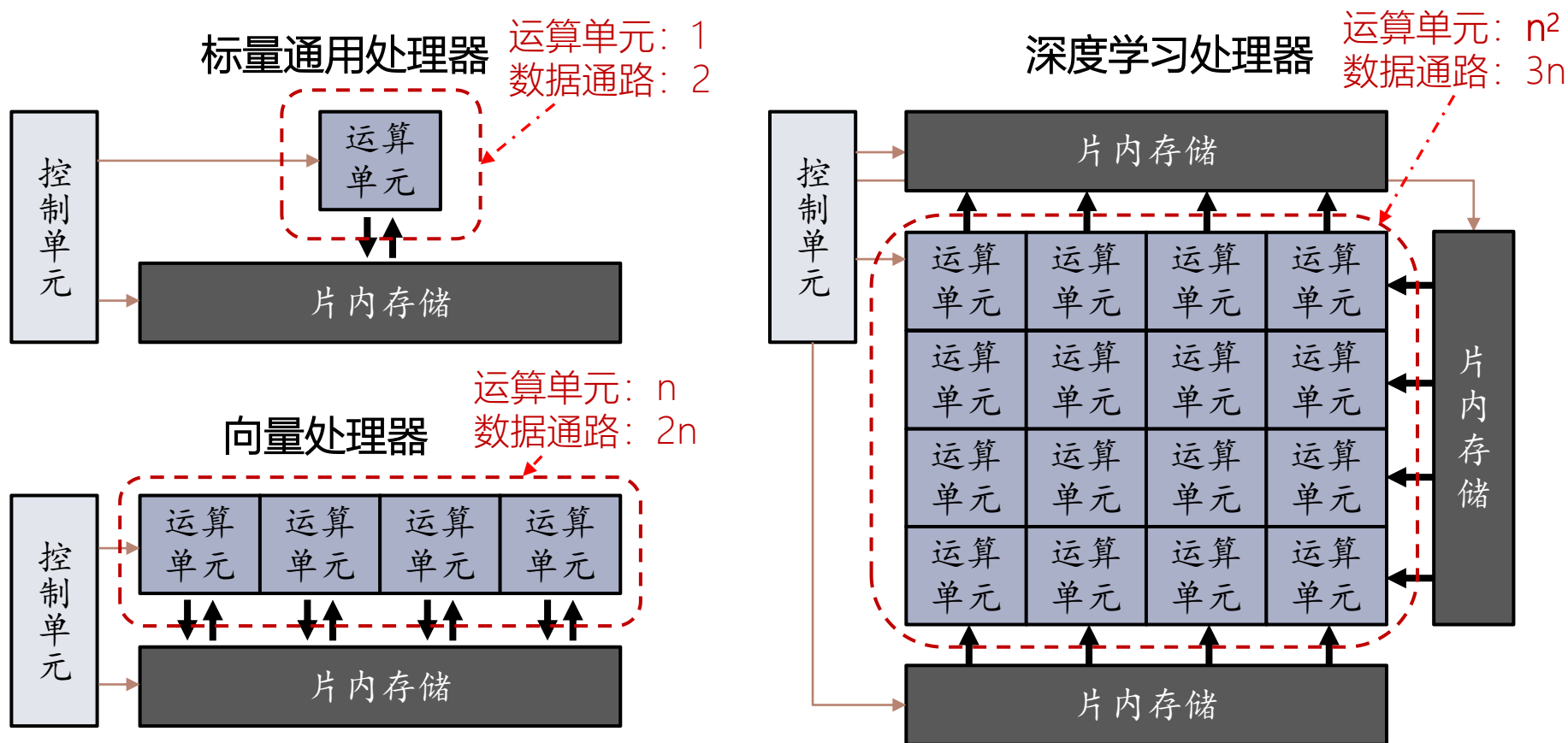
深度学习处理器 (DLP)

基本运算单元从向量运算扩展到矩阵运算



深度学习处理器 (DLP)

基本运算单元从向量运算扩展到矩阵运算



深度学习处理器在相同带宽下, 提供更高运算能力

DLP结构：控制

- 深度学习程序控制流以计数循环为主

```
for(co=0; co<Co; co++) for(yo=0; yo<Ho; yo++) for(xo=0; xo<Wo; xo++) {  
    output[co][yo][xo] = bias[co];  
    for(yk=0; yk<Hk; yk++) for(xk=0; xk<Wk; xk++) for(ci=0; ci<Ci; ci++)  
{  
        output[co][yo][xo] += weight[co][ci][yk][xk]  
            * input[ci][yo*stride+yk][xo*stride+xk];  
    }  
}
```


DLP结构：控制

- 深度学习程序控制流以计数循环为主

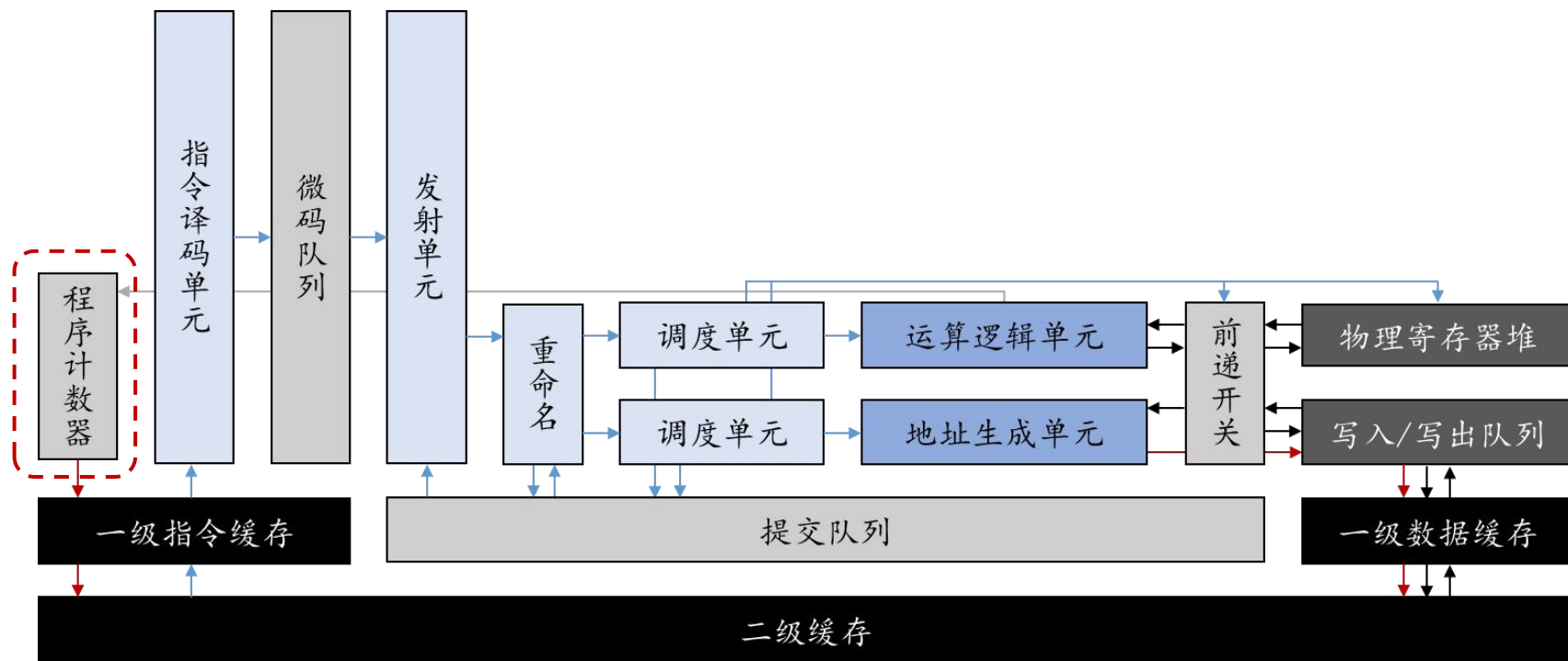
```
for(co=0; co<Co; co++) for(yo=0; yo<Ho; yo++) for(xo=0; xo<Wo; xo++) {  
    output[co][yo][xo] = bias[co];  
    for(yk=0; yk<Hk; yk++) for(xk=0; xk<Wk; xk++) for(ci=0; ci<Ci; ci++)  
{  
        output[co][yo][xo] += weight[co][ci][yk][xk]  
                               * input[ci][yo*stride+yk][xo*stride+xk];  
    }  
}
```

- 启发：设计计数跳转指令

- 大多数跳转无需分支预测

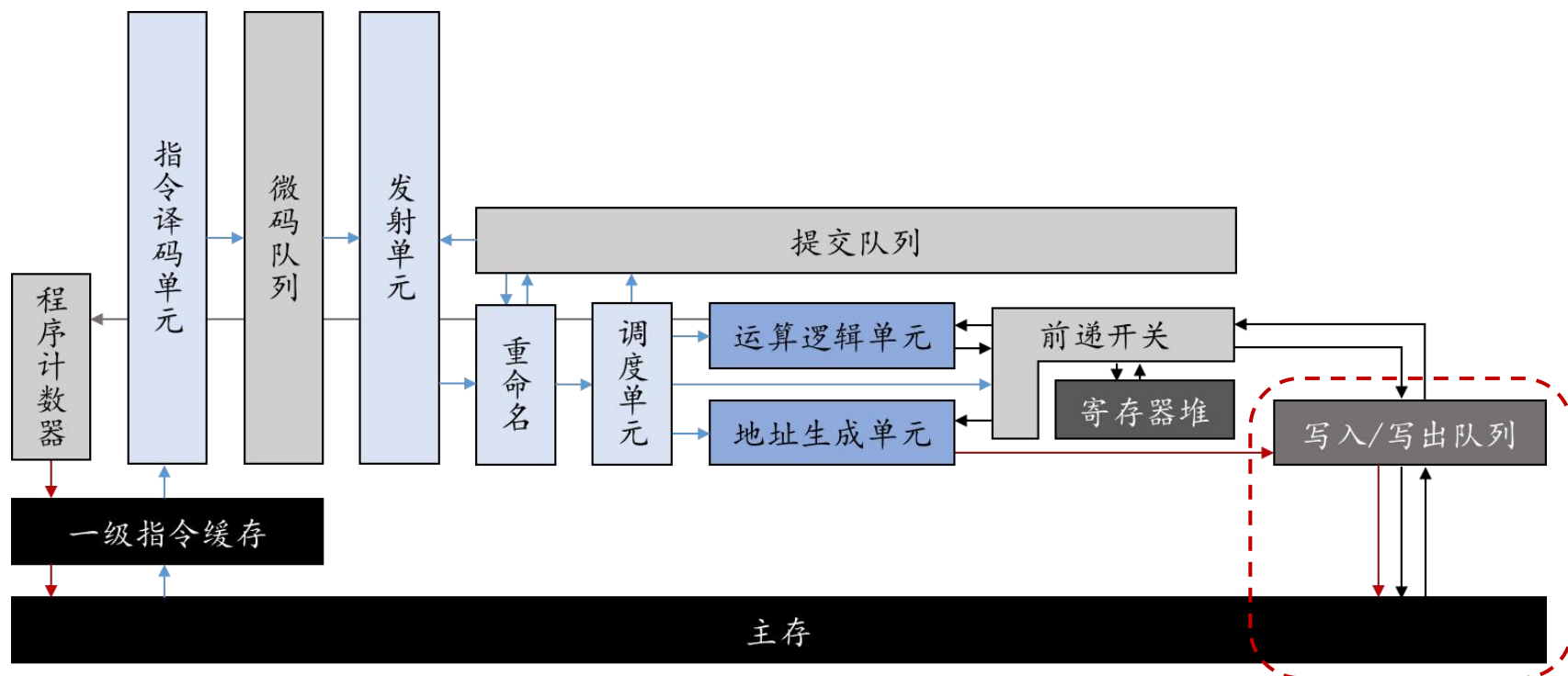
DLP结构

以精简的通用处理器结构为基础



DLP结构

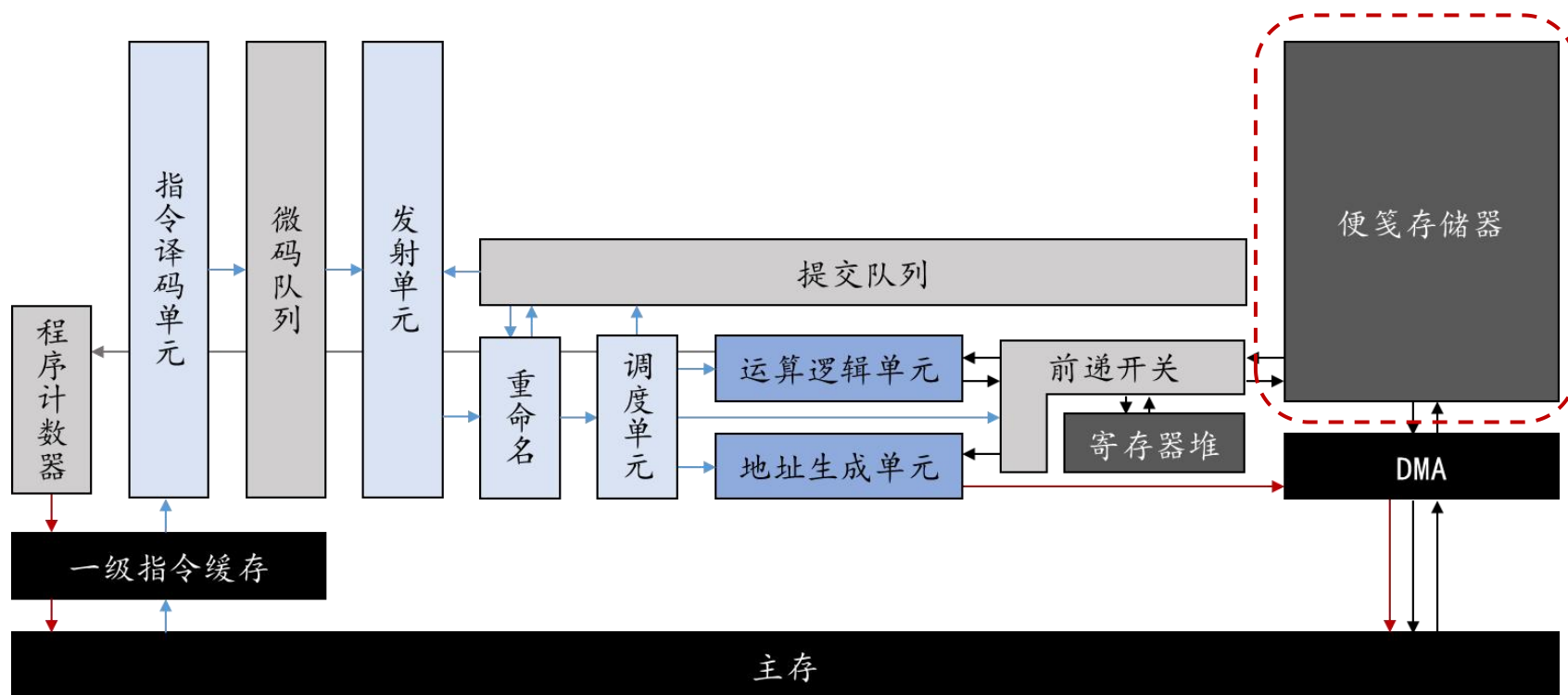
取消数据缓存，I/O直连外部主存



DLP结构

添加**便笺存储器**和**直接访存模块 (DMA)**

DMA：代理主存与便笺存储器之间的大块连续数据搬运



DLP结构：访存

► 缓存相比便笺存储

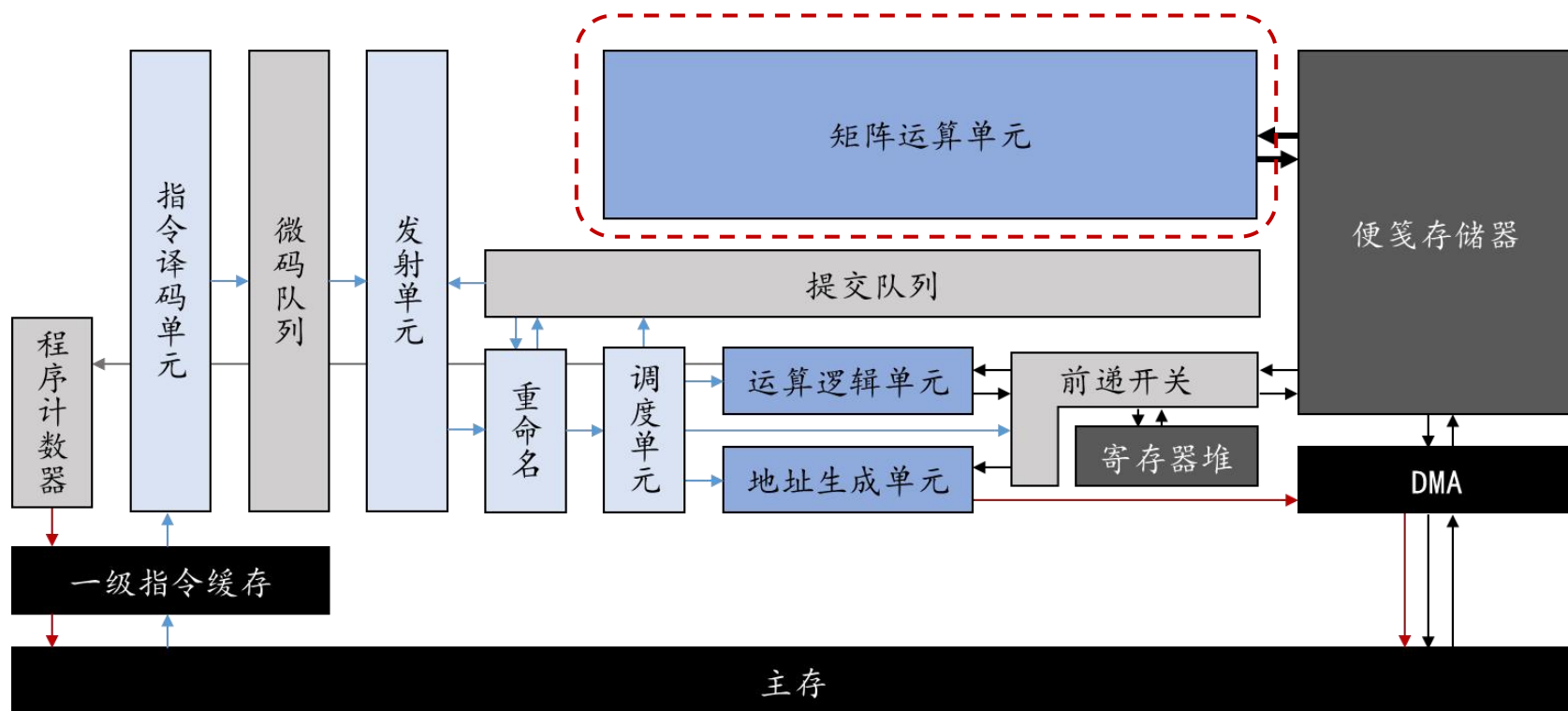
4MB, 7nm FinFET工艺评估

	缓存	便笺
面积	+50%	
速度		+23%
功耗	+2%	
编程	透明	需要控制

► 深度学习访存行为规律，更适合使用便笺存储

DLP结构

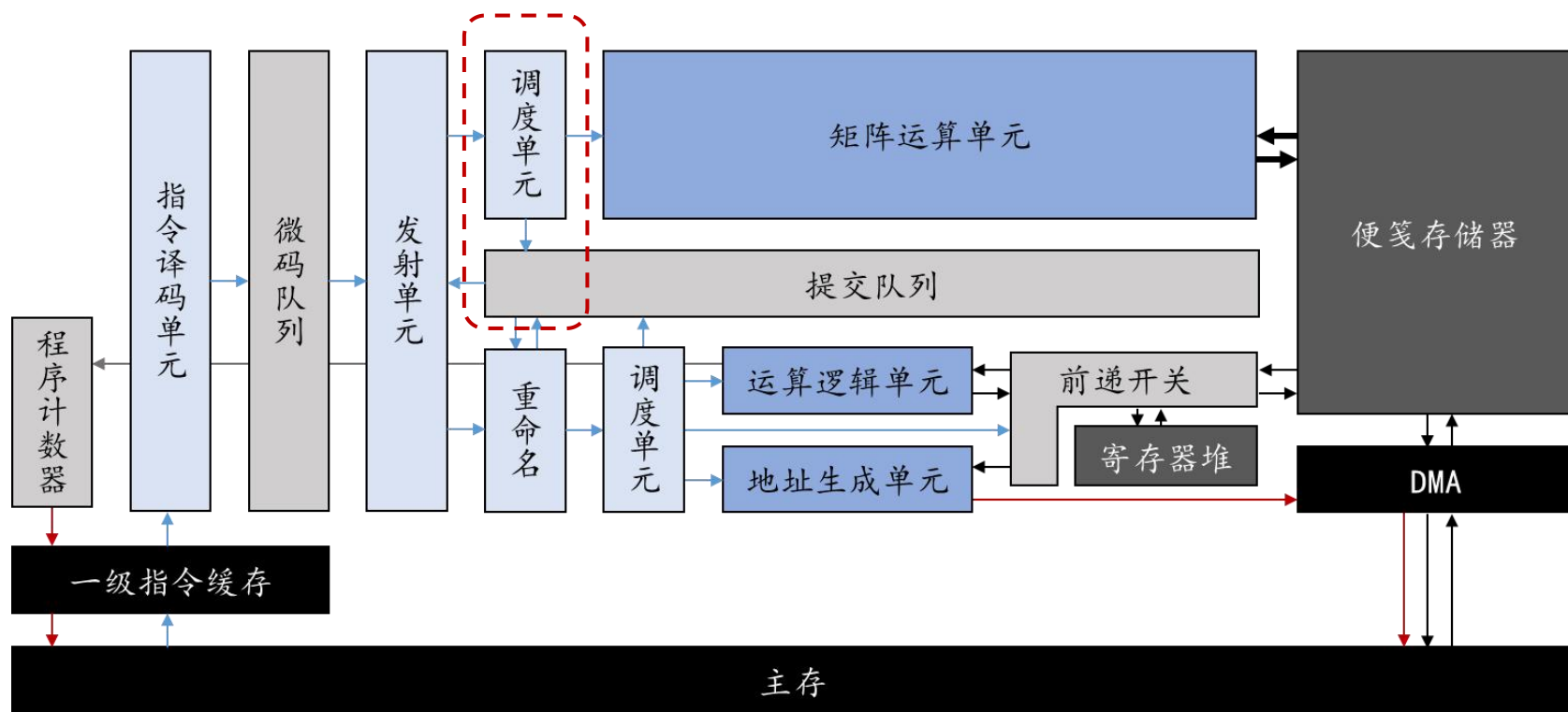
添加矩阵运算单元



DLP结构

添加矩阵指令的控制单元

- ▶ 矩阵指令在提交队列中，与标量/访存指令同步



DLP结构

进一步优化：

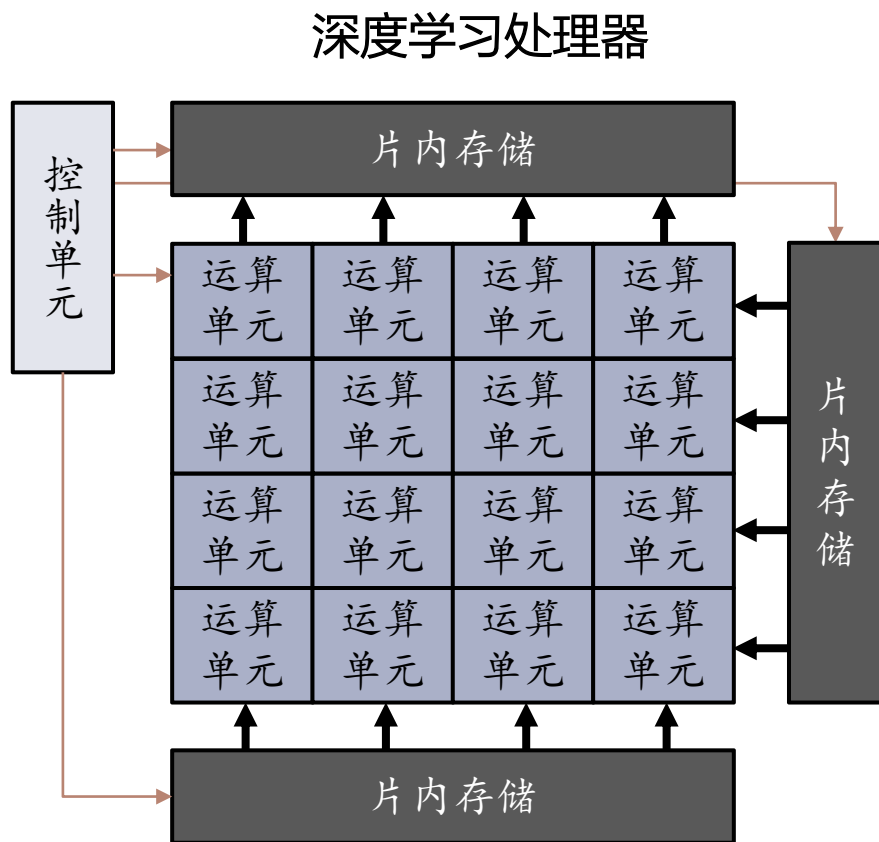
- ▶ 分析：深度学习的指令需要上百、上万时钟周期，执行指令的频率很低，没有必要专门设计结构节约一两个时钟周期
- ▶ 深度学习以计数循环为主要控制结构，条件分支较少，但分支预测器价格不菲，因此可以省去分支预测器
- ▶ 数据前递开关、寄存器重命名、多发射等结构同理，对深度学习处理器的性能影响较小，也可以去掉

讨论

- ▶ 深度学习处理器的优势：
 - ▶ 一条指令完成复杂的线性运算，控制开销低
 - ▶ 同等访存，能达到更强运算能力
- ▶ 缺陷是什么？

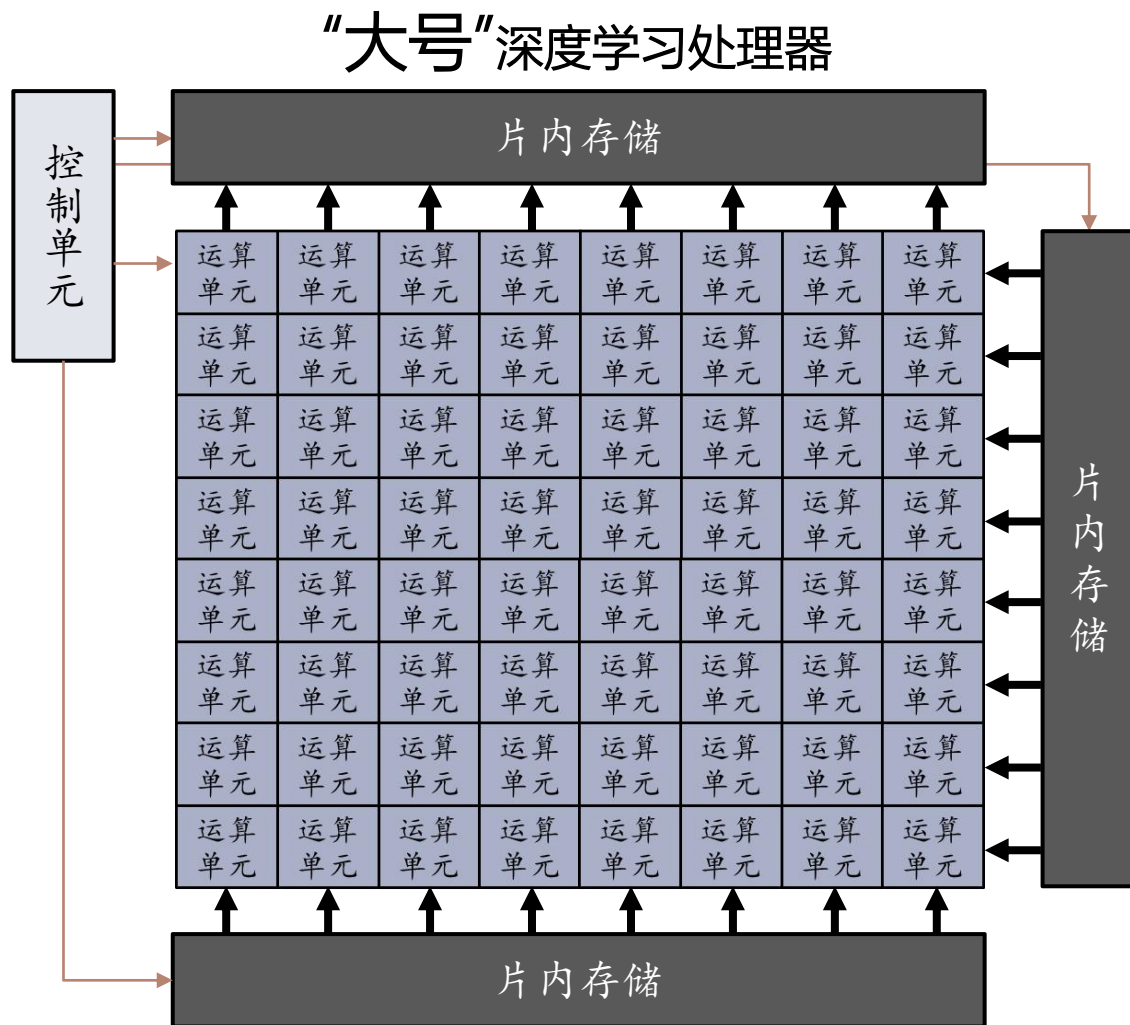
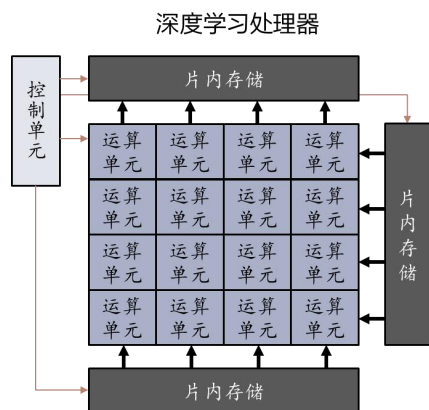
DLP的规模扩展

- 如何堆积更强的运算能力？



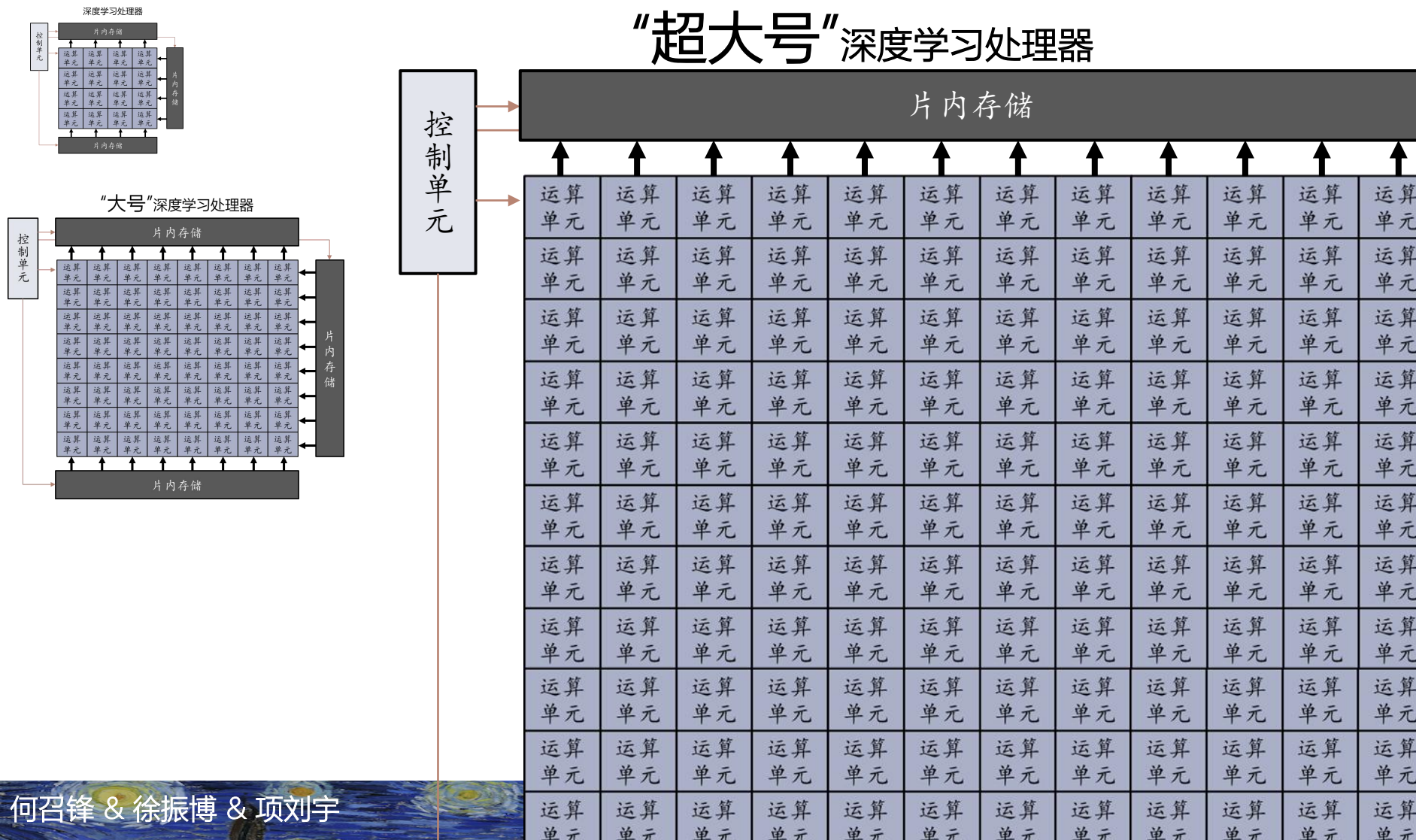
DLP的规模扩展

► 如何堆积更强的运算能力？



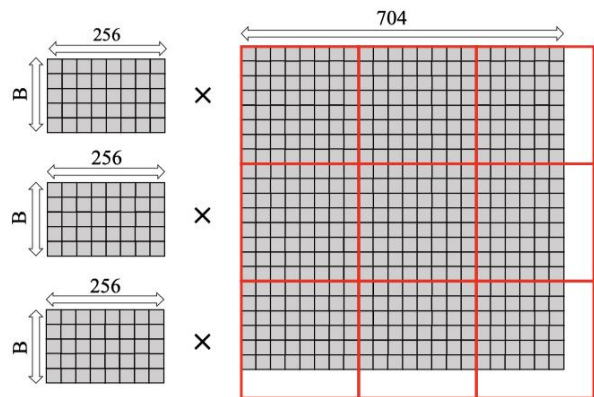
DLP的规模扩展

如何堆积更强的运算能力？

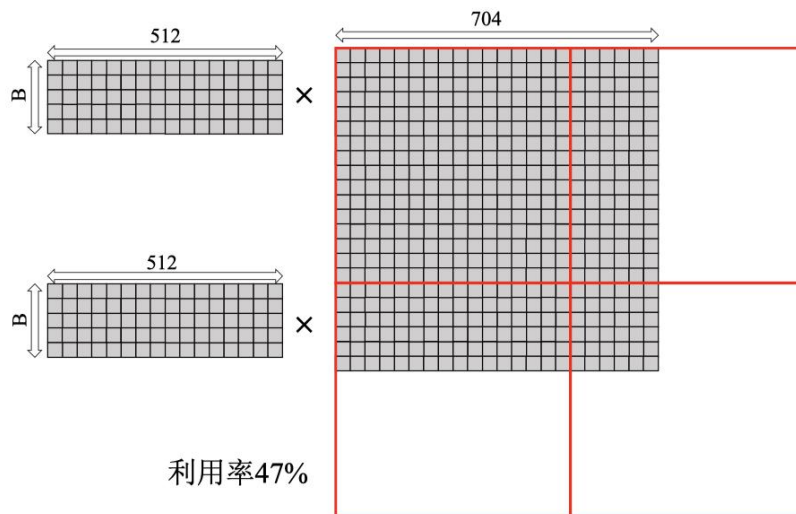


利用率问题

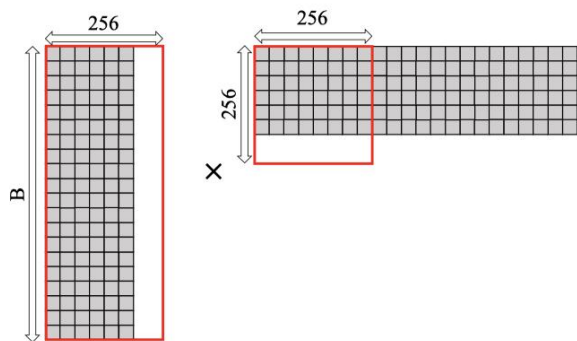
- 简单放大矩阵运算单元，导致利用率低下



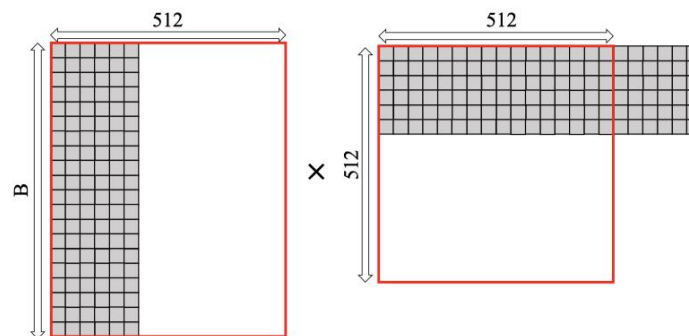
利用率84%



利用率47%



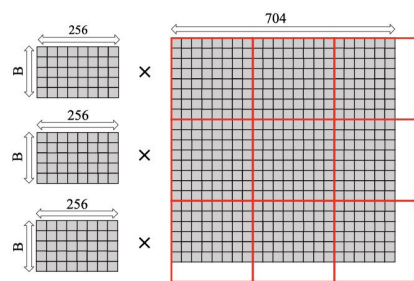
利用率56%



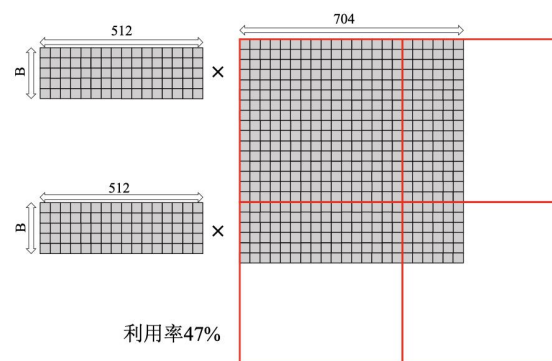
利用率14%

利用率问题

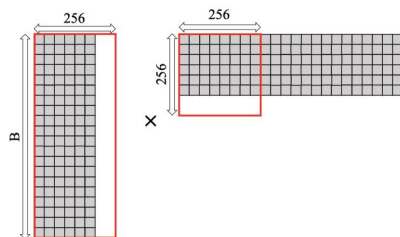
- ▶ 模数效应：虽然矩阵运算规模扩大具有更好的理论运算密度，但是算法规模跟硬件规模不能整除的时候，需要补零，导致运算器、寄存器、主存、DMA等利用率降低
- ▶ 运算器规模越大，模数效应越严重



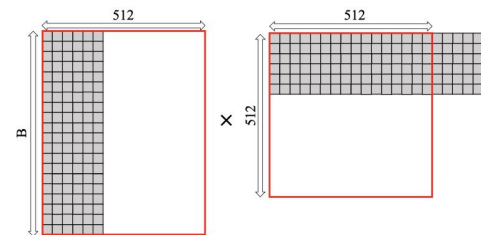
利用率84%



利用率47%



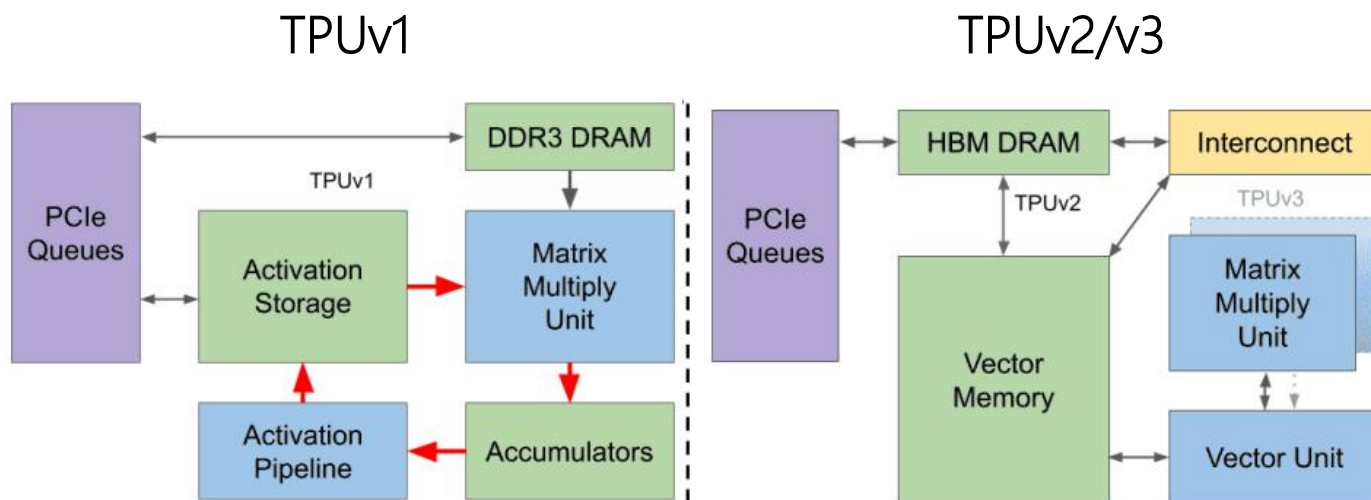
利用率56%



利用率14%

教训：TPU

- ▶ TPUv1 (2016) : 256×256
- ▶ TPUv2 (2017) : 128×128
- ▶ TPUv3 (2018) : $128 \times 128 + 128 \times 128$



N. P. Jouppi et al., "Ten Lessons From Three Generations Shaped Google's TPUv4i : Industrial Product," 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA), Valencia, Spain, 2021

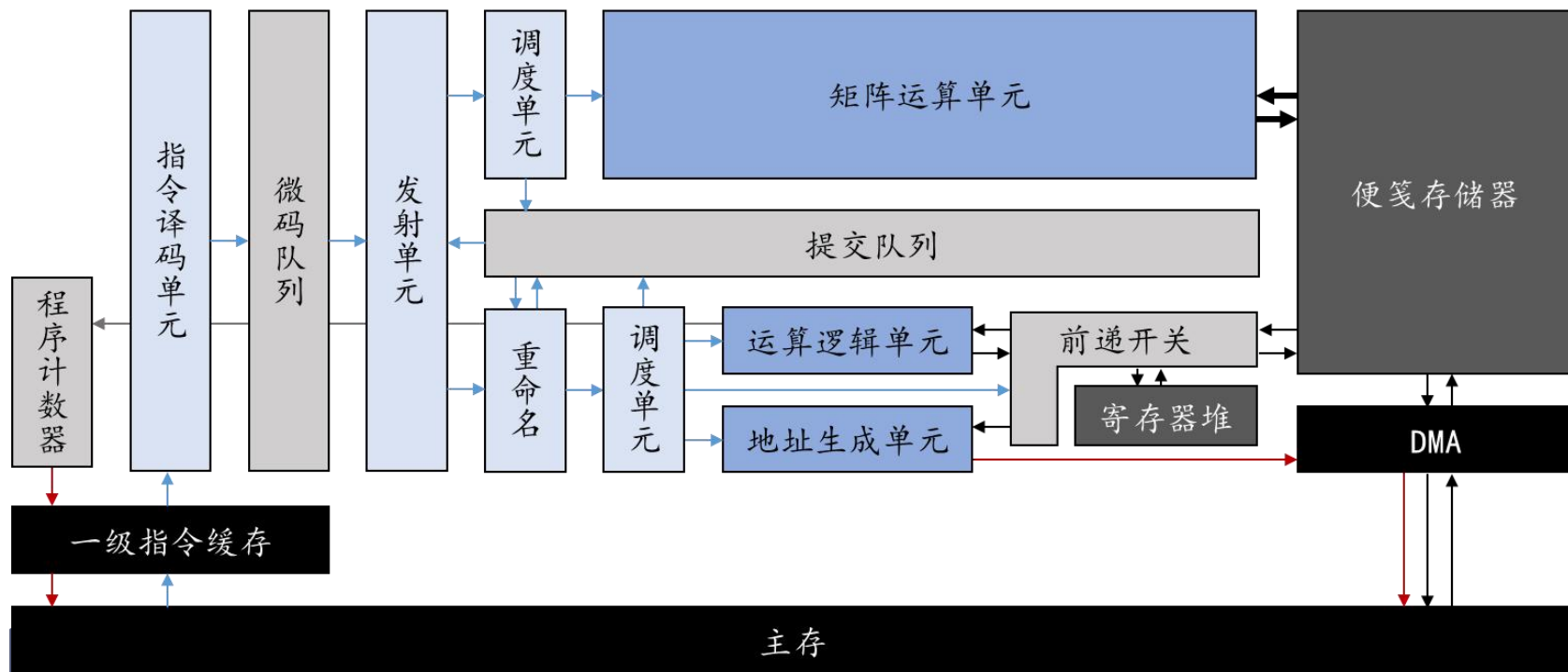
多核DLP

相比单体、巨大的矩阵运算单元，

- ▶ 分立的多个较小矩阵运算单元运用更灵活
 - ▶ 但相应地，增加硬件成本（特别是带宽）
 - ▶ 按需设计！

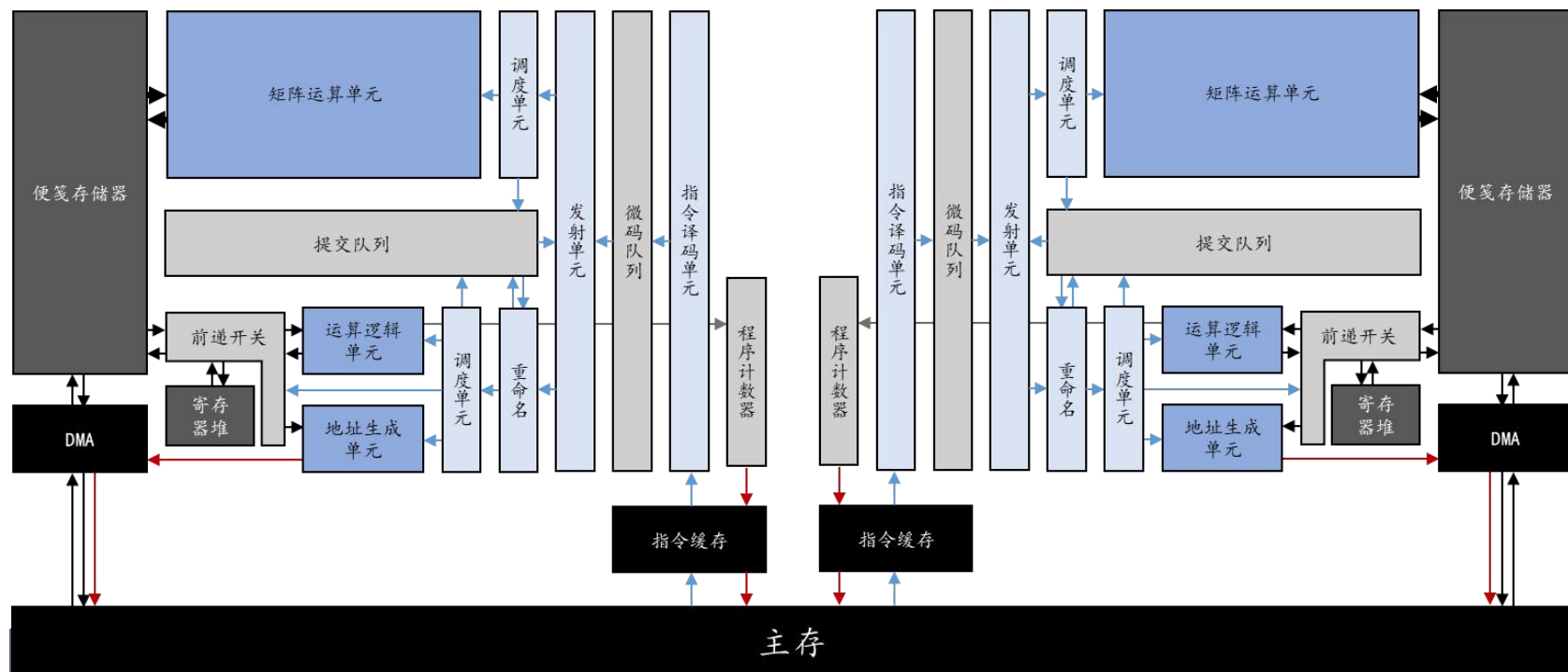
多核DLP结构

以前述DLP为基础



多核DLP结构

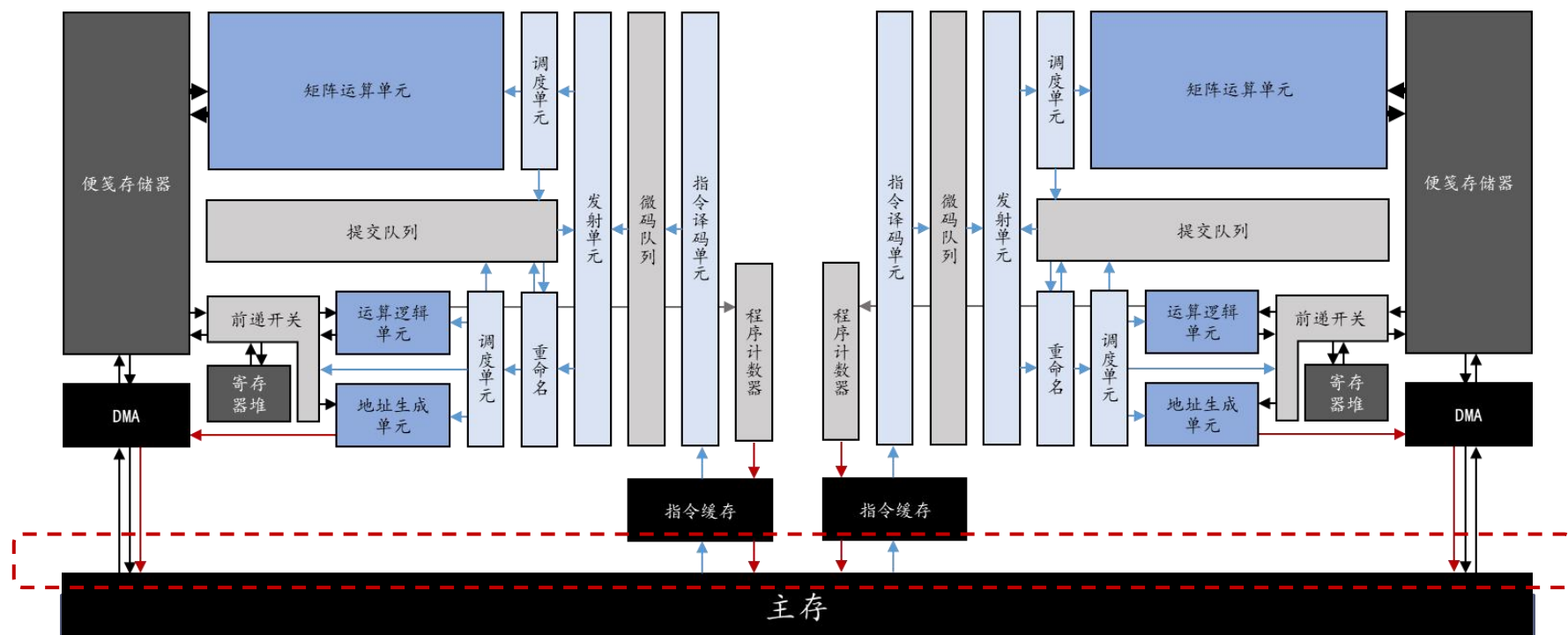
将核心设计复制一份？



多核DLP结构

将核心设计复制一份？

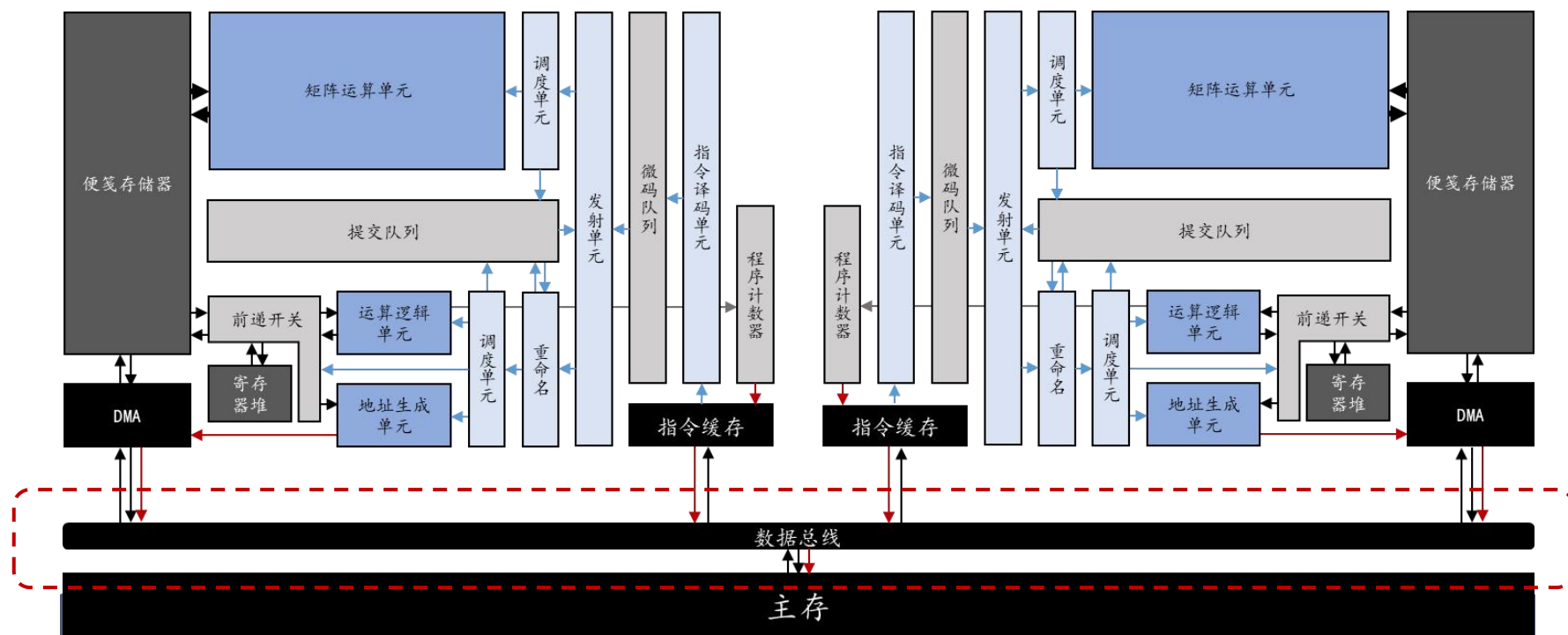
► 问题：要求外部主存提供的接口翻倍了



多核DLP结构

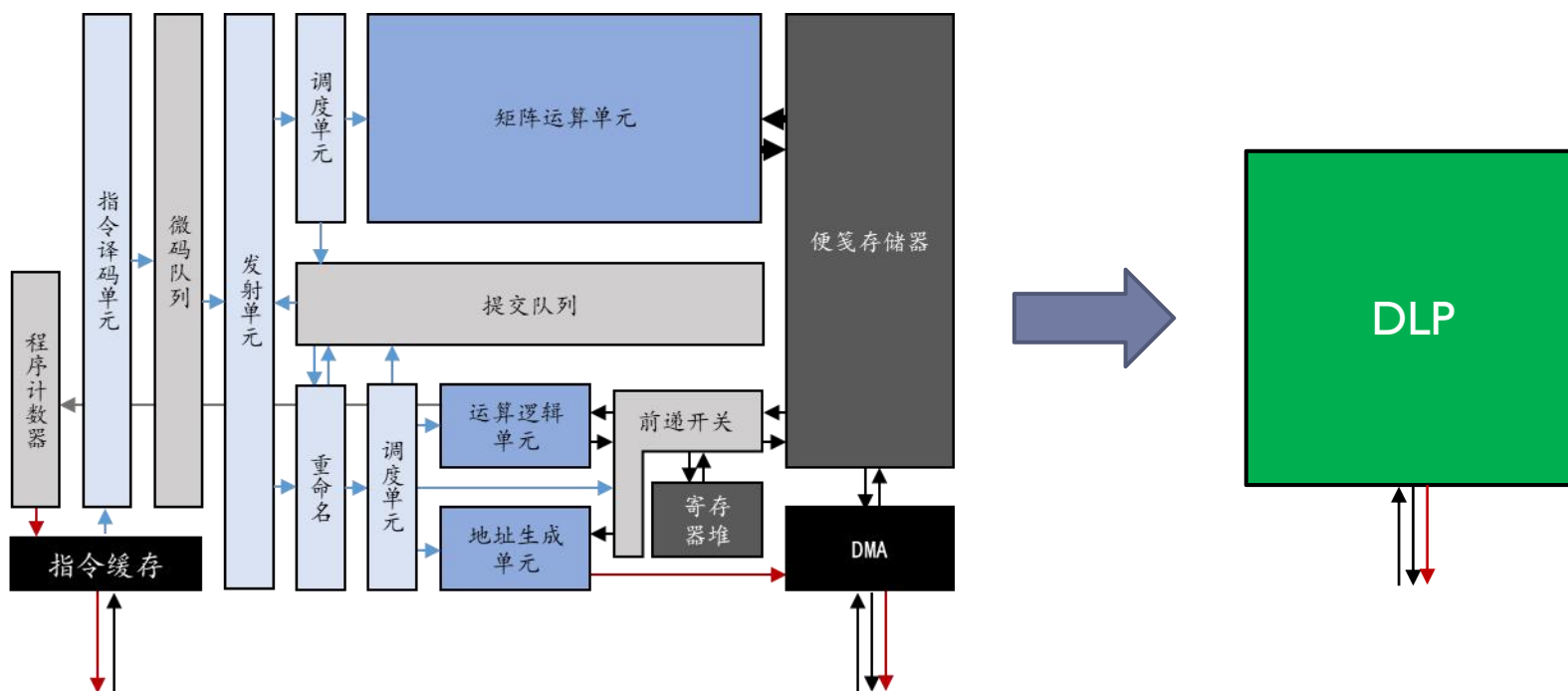
采用总线结构，共享主存数据通路

- 发生争用时，总线进行**仲裁**，决定数据通路“通行权”



多核DLP结构

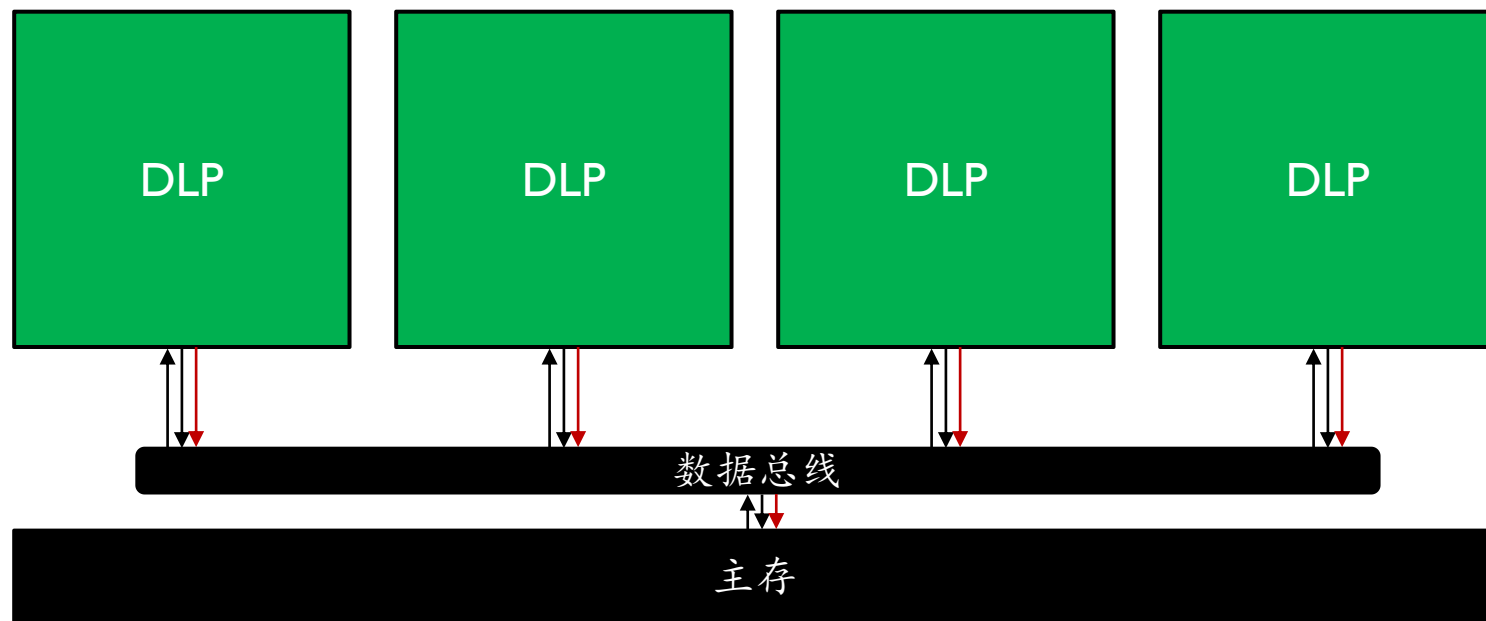
将一份完整的DLP结构视为一个核



多核DLP结构

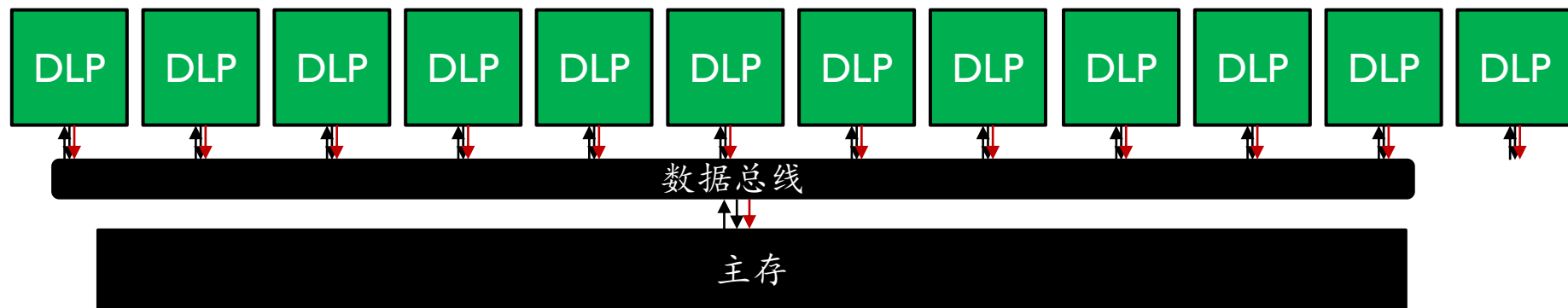
一致内存访问 (UMA) 模型

—— “一字长蛇阵”



多核DLP结构

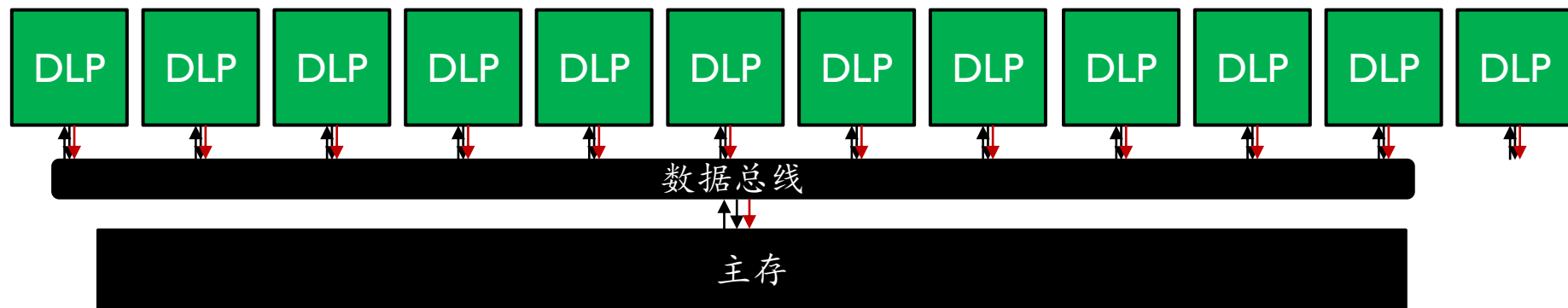
“一字长蛇阵” 可以无限长吗？



多核DLP结构

“一字长蛇阵” 可以无限长吗？

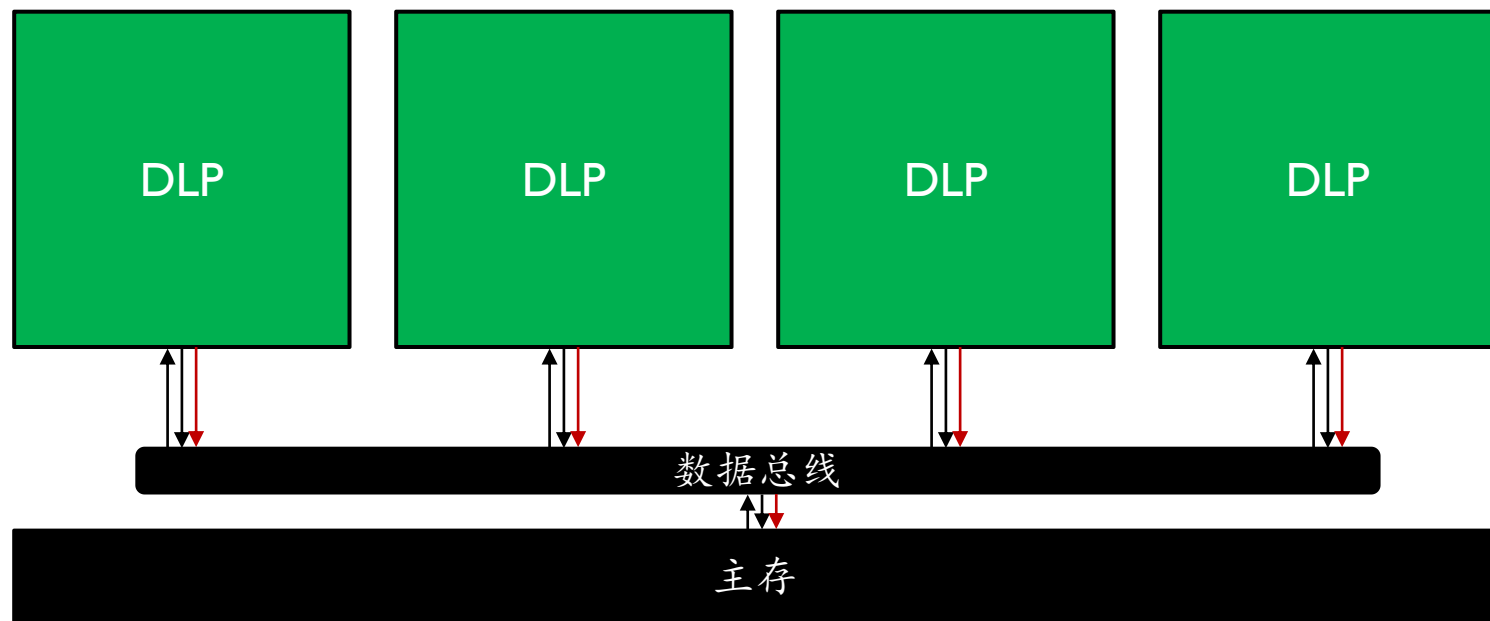
► 总线争用将成为瓶颈



多核DLP结构

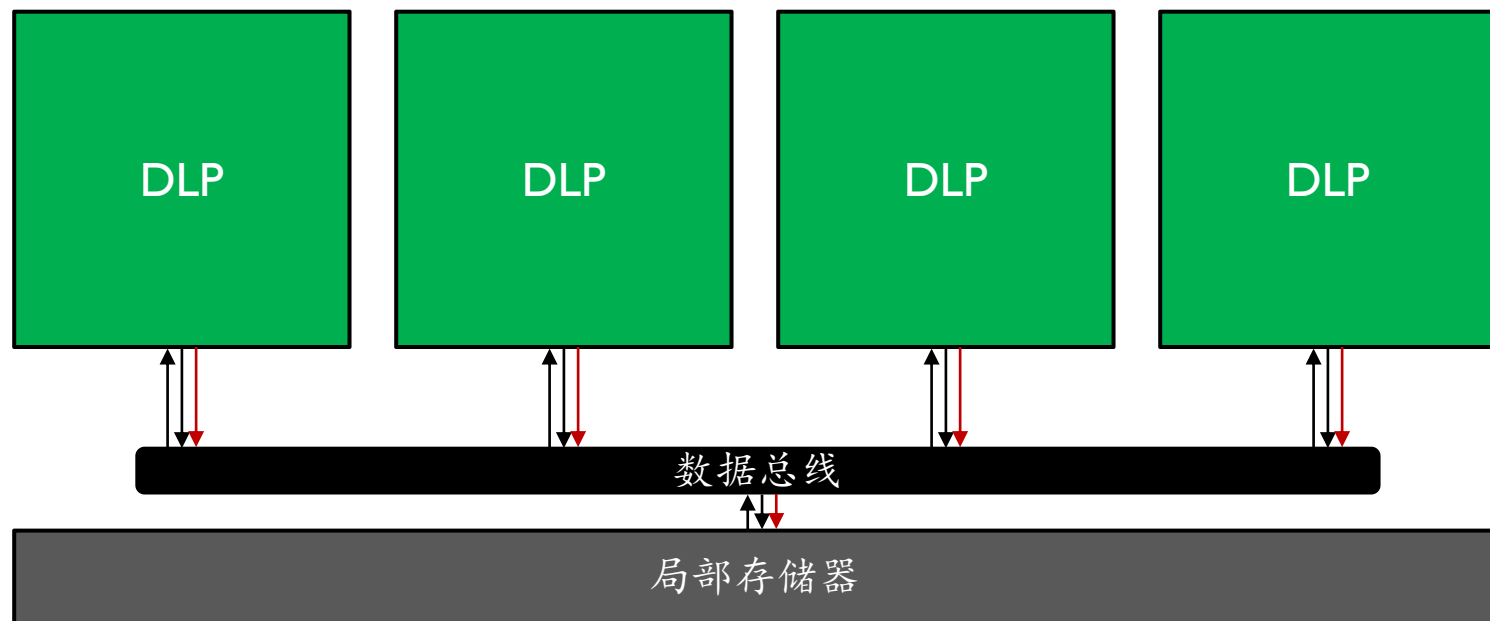
以UMA多核为基础

思想：把DLP当作一个矩阵运算器



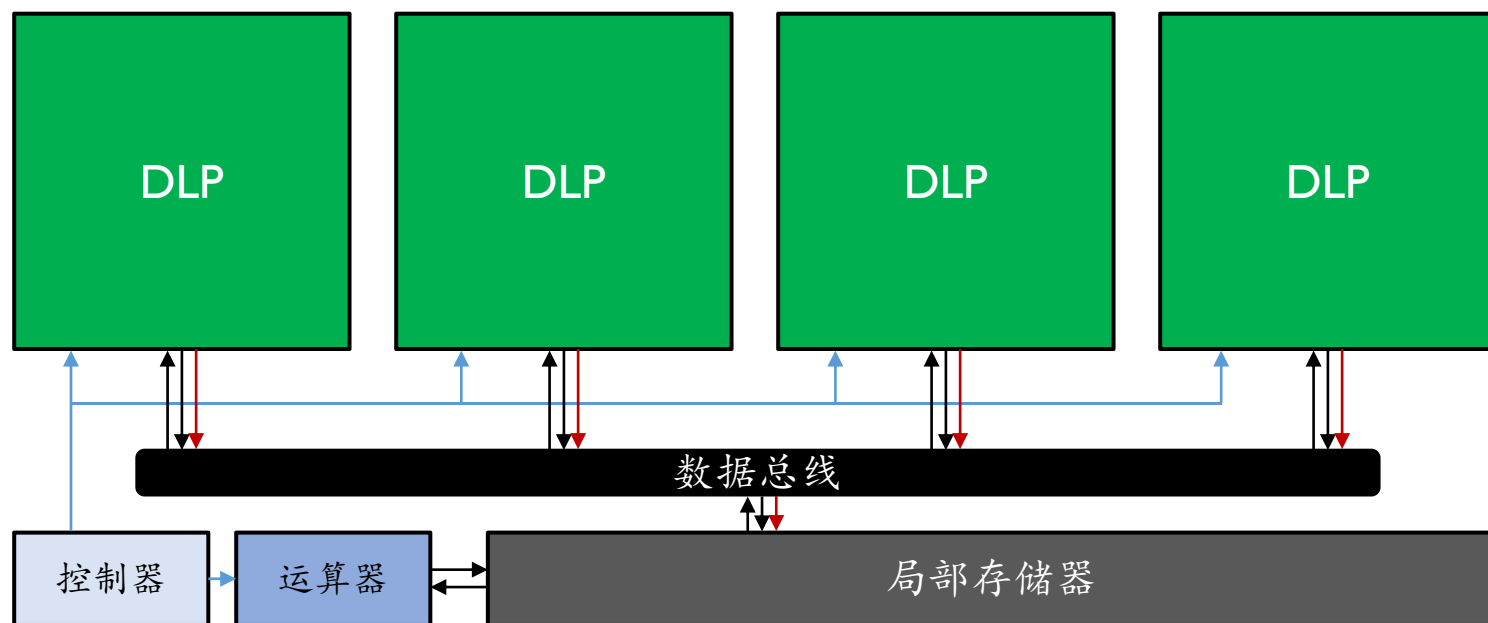
多核DLP结构

添加局部存储器



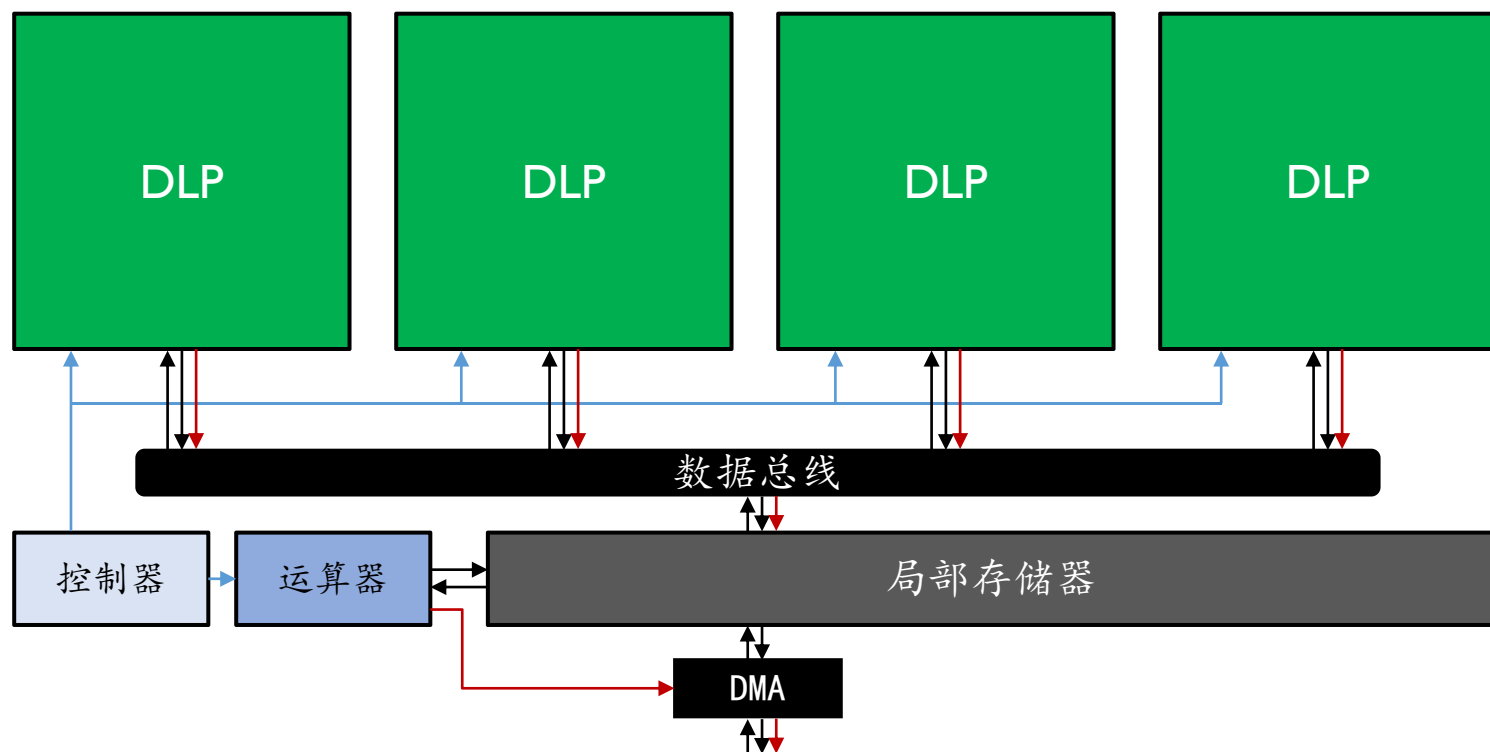
多核DLP结构

添加局部控制器、运算器



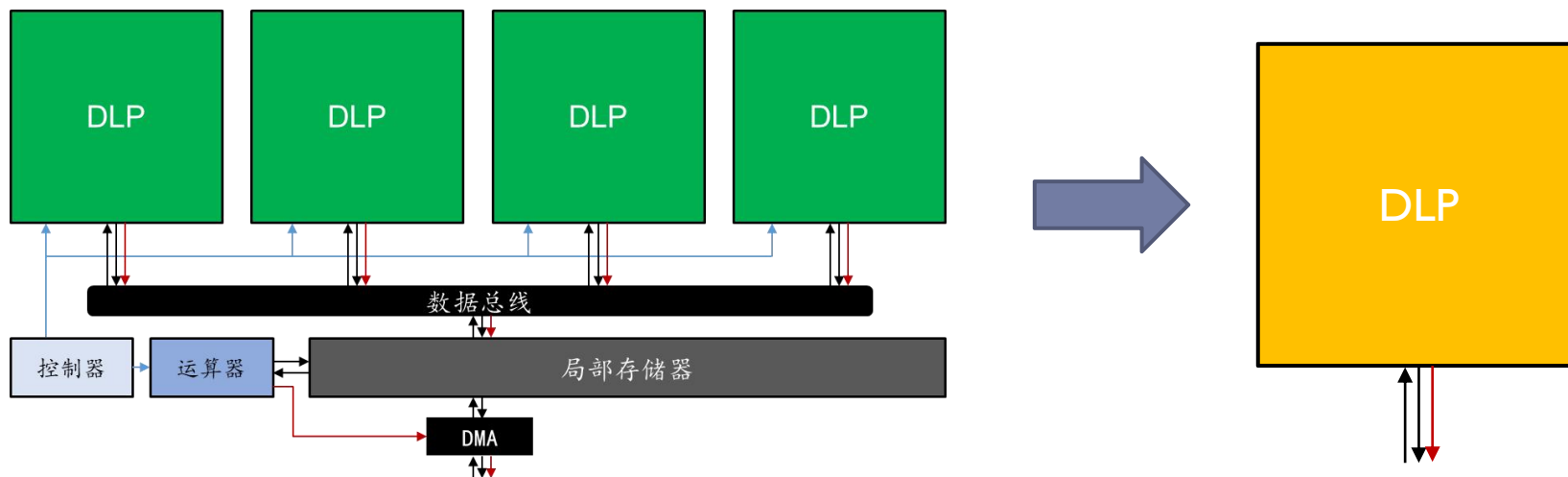
多核DLP结构

添加局部DMA，与外部存储相连



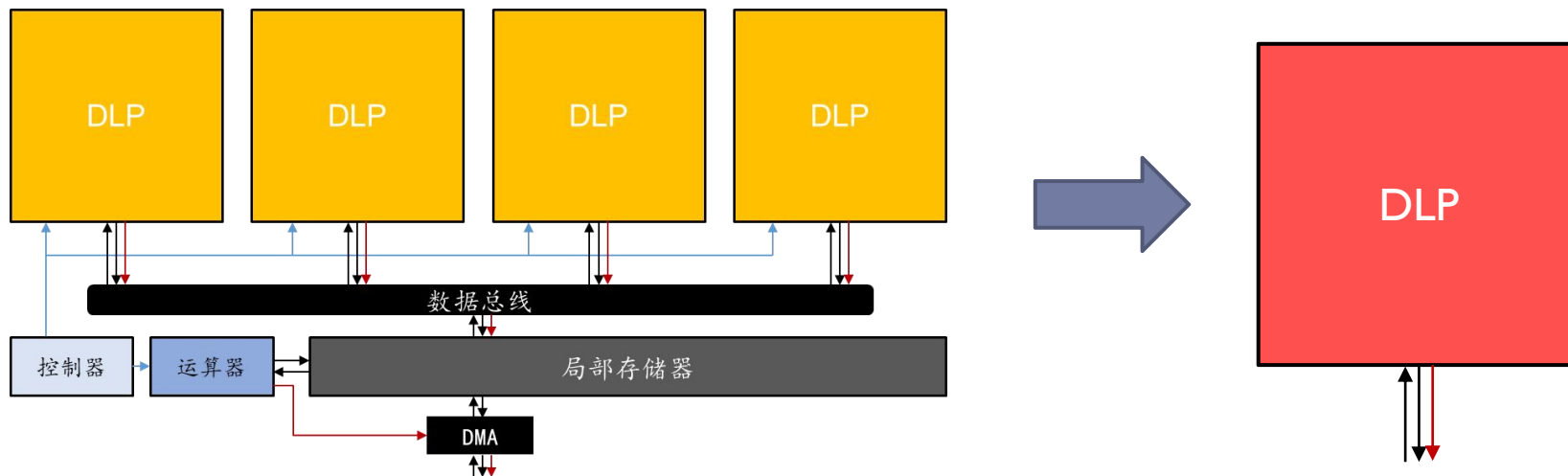
多核DLP结构

将一份完整的UMA多核DLP结构，视为一个核



多核DLP结构

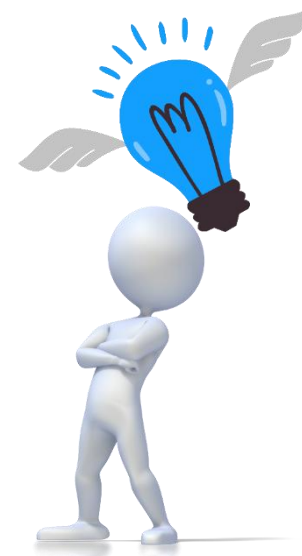
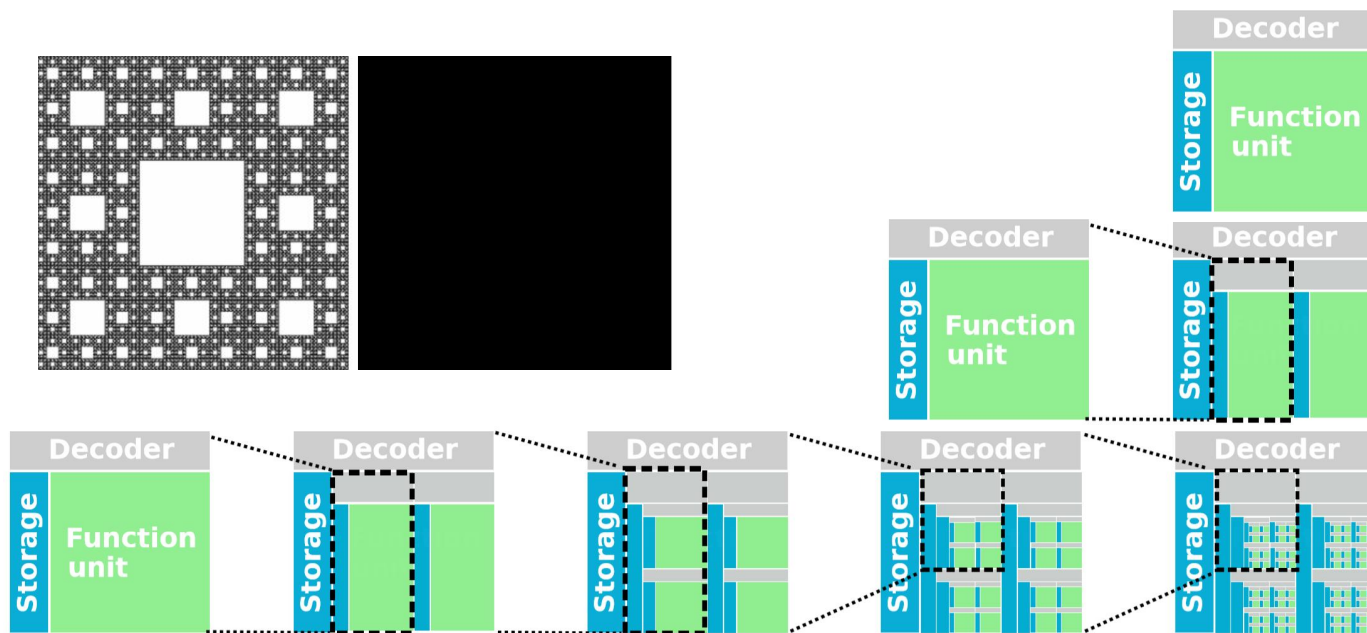
将一份完整的UMA多核DLP结构，视为一个核



大规模深度学习处理器

分形DLP：以相同规则任意扩展的DLP结构

- ▶ 同一份分治程序运行在所有的局部控制器上
- ▶ 无论DLP规模多大，在使用者的视角是一样的！



Yongwei Zhao, Zidong Du, Qi Guo, Shaoli Liu, Ling Li, Zhiwei Xu, Tianshi Chen, Yunji Chen: Cambricon-F: machine learning computers with fractal von neumann architecture. ISCA 2019.

总结

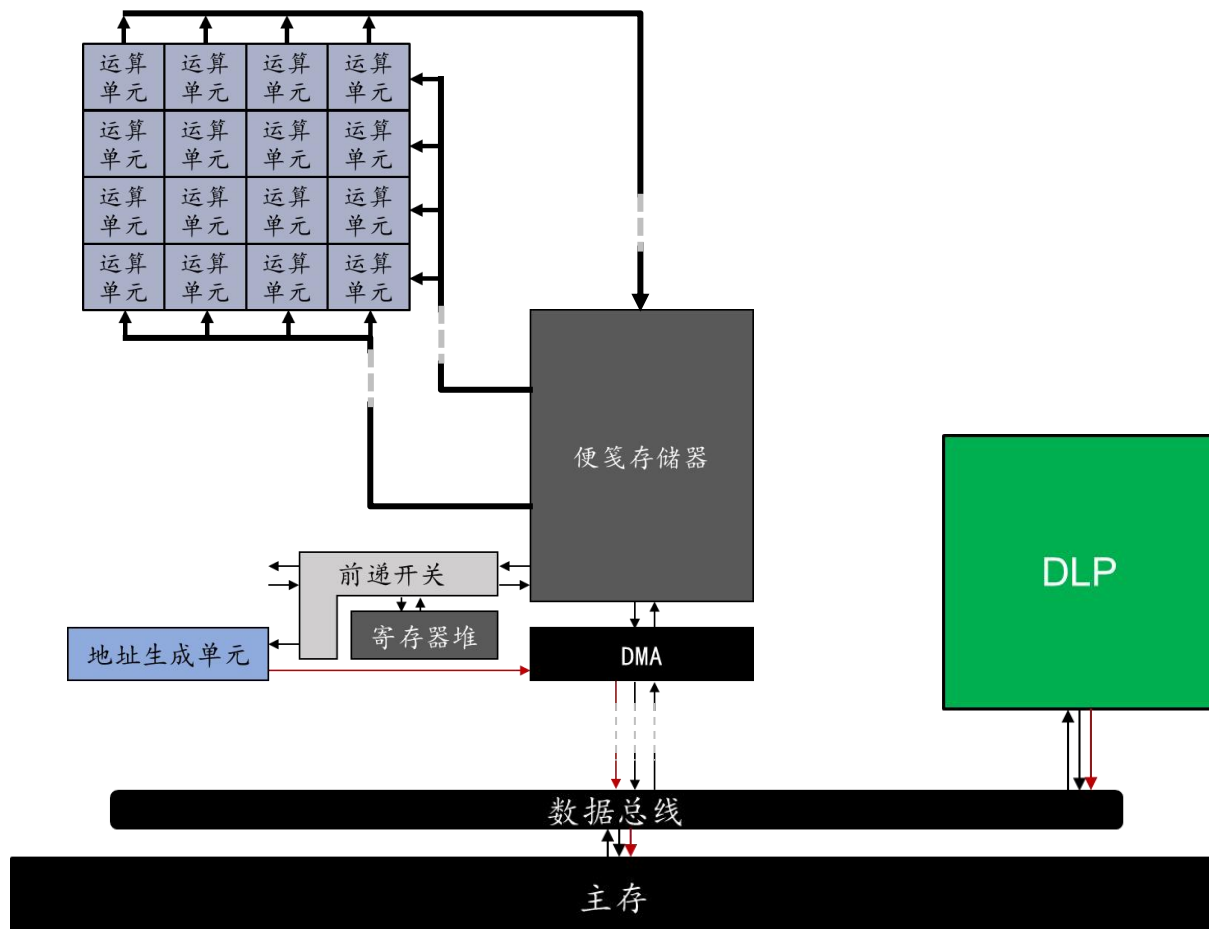
- ▶ 系统设计需要根据实际情况，按需确定
- ▶ 深度学习处理器（DLP）的特点
 - ▶ 计算：矩阵、卷积运算为主，辅以标量、向量运算
 - ▶ 控制：计数循环为主，辅以条件分支指令
 - ▶ 存储：便笺存储器为主，辅以寄存器
 - ▶ 采用分形方式进行规模扩展
- ▶ 深度学习处理器（DLP）的基本原理
 - ▶ 改善基本运算的I/O复杂度

总体架构

► 计算

► 访存

► 通信





谢谢大家！